02. World Chain 아키텍처 분석

개요

World Chain은 OP Stack을 기반으로 구축된 블록체인으로, Priority Blockspace for Humans (PBH) 메커니즘을 통해 인증된 사용자에게 우선순위를 제공합니다. 이 문서에서는 World Chain의 전체 아키텍처와 OP Stack과의 통합 방식, 그리고 rollup-boost를 통한 외부 블록 빌더 지원에 대해 상세히 분석합니다.

2.1 OP Stack 기반 구조 분석 및 설명

OP Stack 기본 구조

World Chain은 Optimism의 OP Stack을 기반으로 구축되어 있으며, 다음과 같은 핵심 구성 요소들을 활용합니다:

실행 계층 (Execution Layer)

- **reth 기반 실행 클라이언트**: World Chain은 Paradigm의 reth를 기반으로 한 실행 클라이언트를 사용합니다
- **OpEvmConfig**: Optimism 특화 EVM 설정을 통해 L2 특성을 지원합니다
- **OpChainSpec**: Optimism 체인 스펙을 확장하여 World Chain 고유의 설정을 포함합니다

```
// World Chain의 노드 타입 정의

impl<T: Unpin + Send + Clone + Sync + Debug + 'static> NodeTypes for WorldChainN

type Primitives = OpPrimitives;

type ChainSpec = OpChainSpec;
```

```
type StateCommitment = MerklePatriciaTrie;

type Storage = OpStorage;

type Payload = OpEngineTypes;

}
```

합의 계층 (Consensus Layer)

- **OpConsensusBuilder**: Optimism의 합의 메커니즘을 구현합니다
- **Engine API**: 합의 계층과 실행 계층 간의 통신을 담당합니다

World Chain의 OP Stack 확장 방식

World Chain은 OP Stack의 기본 구조를 유지하면서 다음과 같은 확장을 제공합니다:

1. 커스텀 트랜잭션 풀

```
pub  type WorldChainTransactionPool<Provider, BlobStore> =

reth_transaction_pool::Pool<

WorldChainTransactionValidator<OpTransactionValidator<...>>,

WorldChainOrdering,

BlobStore

>;
```

- **WorldChainTransactionValidator**: PBH 트랜잭션 검증 로직을 포함합니다
- **WorldChainOrdering**: PBH 우선순위 정책에 따른 트랜잭션 정렬을 구현합니다
- **WorldChainRootValidator**: World ID 루트 검증을 담당합니다

2. 커스텀 페이로드 빌더

```rust
pub struct WorldChainPayloadBuilder<Provider, BlobStore, Txs> {
// OP Stack 기본 구성 요소들
pool: WorldChainTransactionPool<Provider, BlobStore>,
provider: Provider,
evm_config: OpEvmConfig,
// World Chain 특화 설정들
verified_blockspace_capacity: u8,
pbh_entry_point: Address,
pbh_signature_aggregator: Address,
builder_private_key: String,
}
```

Engine API와 블록 생성 과정

World Chain의 블록 생성 과정은 표준 OP Stack Engine API를 따르면서 PBH 로직을 통합합니다:

블록 생성 단계별 분석

1. **Fork Choice Update**: 합의 계층이 새로운 블록 생성을 요청합니다

2. **Payload Attributes 처리**: World Chain은 표준 OP Stack 속성에 PBH 관련 정보를 추가로 처리합니다

3. **트랜잭션 선택**: WorldChainOrdering을 통해 PBH 우선순위에 따라 트랜잭션을 정렬합니다

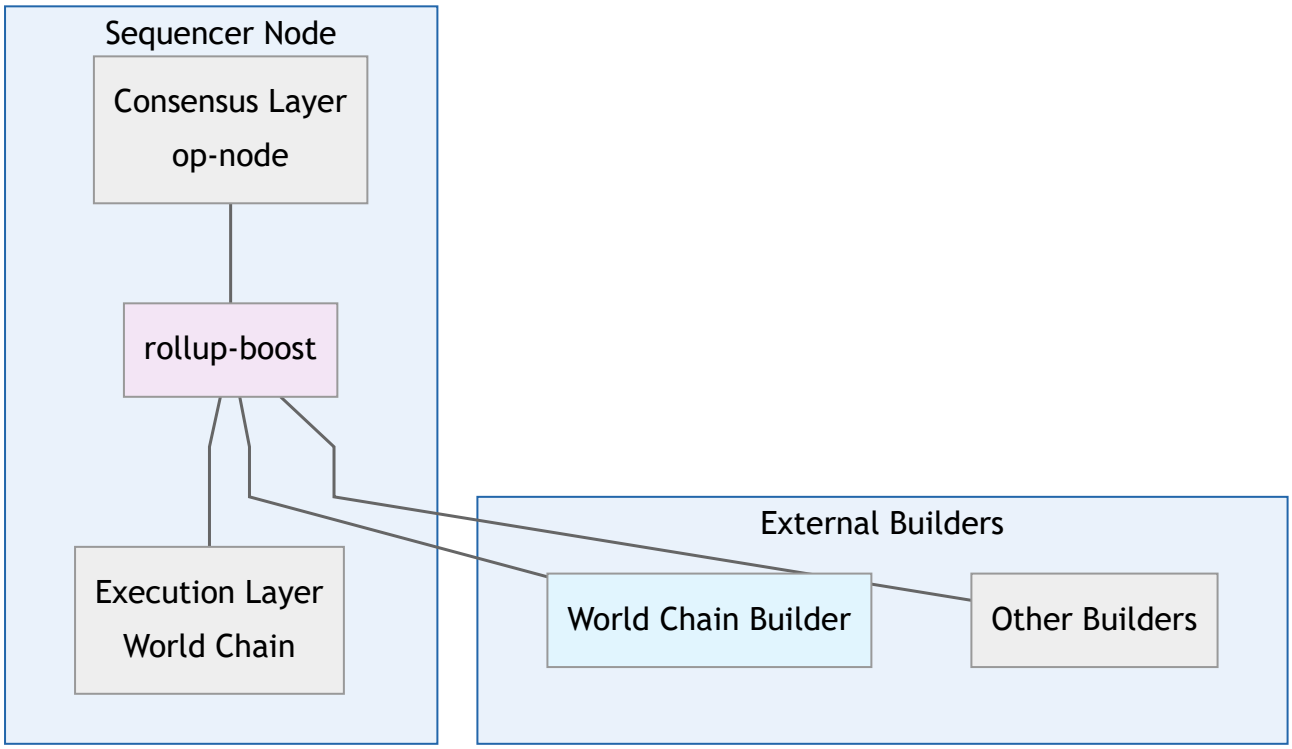
4. **블록 구성**: 지정된 PBH 블록스페이스 용량에 따라 블록을 구성합니다

5. **검증 및 반환**: 구성된 블록을 검증하고 합의 계층에 반환합니다

rollup-boost 통합 방식과 동작 원리

World Chain은 Flashbots의 rollup-boost를 통해 외부 블록 빌더 지원을 구현합니다:

rollup-boost 아키텍처



통합 방식

1. **Engine API 프록시**: rollup-boost는 Engine API 호출을 여러 빌더에게 멀티플렉싱합니다

2. **RPC 프록시**: 모든 RPC 호출을 로컬 실행 클라이언트로 프록시합니다

3. **Miner API 전달**: 가스 가격, 추가 데이터, DA 스로틀링 요청을 외부 빌더에게 전달합니다

4. **트랜잭션 전달**: `eth_sendRawTransaction` 호출을 외부 빌더에게 전달합니다

Flashblocks 통합

World Chain은 flashblocks-p2p를 통해 고성능 블록 빌딩을 지원합니다:

```
pub struct FlashblocksPayloadBuilder<Pool, Client, CtxBuilder, Txs> {
pub evm_config: OpEvmConfig,
pub pool: Pool,
pub client: Client,
pub config: OpBuilderConfig,
pub best_transactions: Txs,
pub ctx_builder: CtxBuilder,
}
```

주요 특징:

- **병렬 블록 빌딩**: 여러 블록을 동시에 구성할 수 있습니다
- **캐시된 읽기**: 이전 빌드의 상태 읽기를 재사용합니다
- **점진적 업데이트**: 새로운 트랜잭션만 추가하여 블록을 업데이트합니다

노드 컨텍스트 시스템

World Chain은 두 가지 운영 모드를 지원합니다:

Basic Context

```
NodeContextType::Basic => {
let node = WorldChainNode::<BasicContext>::new(node_config.clone());
// 기본 World Chain 기능만 활성화
```

```
}
```

Flashblocks Context

```
NodeContextType::Flashblocks => {
let  node = WorldChainNode::<FlashblocksContext>::new(node_config);
// 고성능 블록 빌딩 기능 활성화
}
```

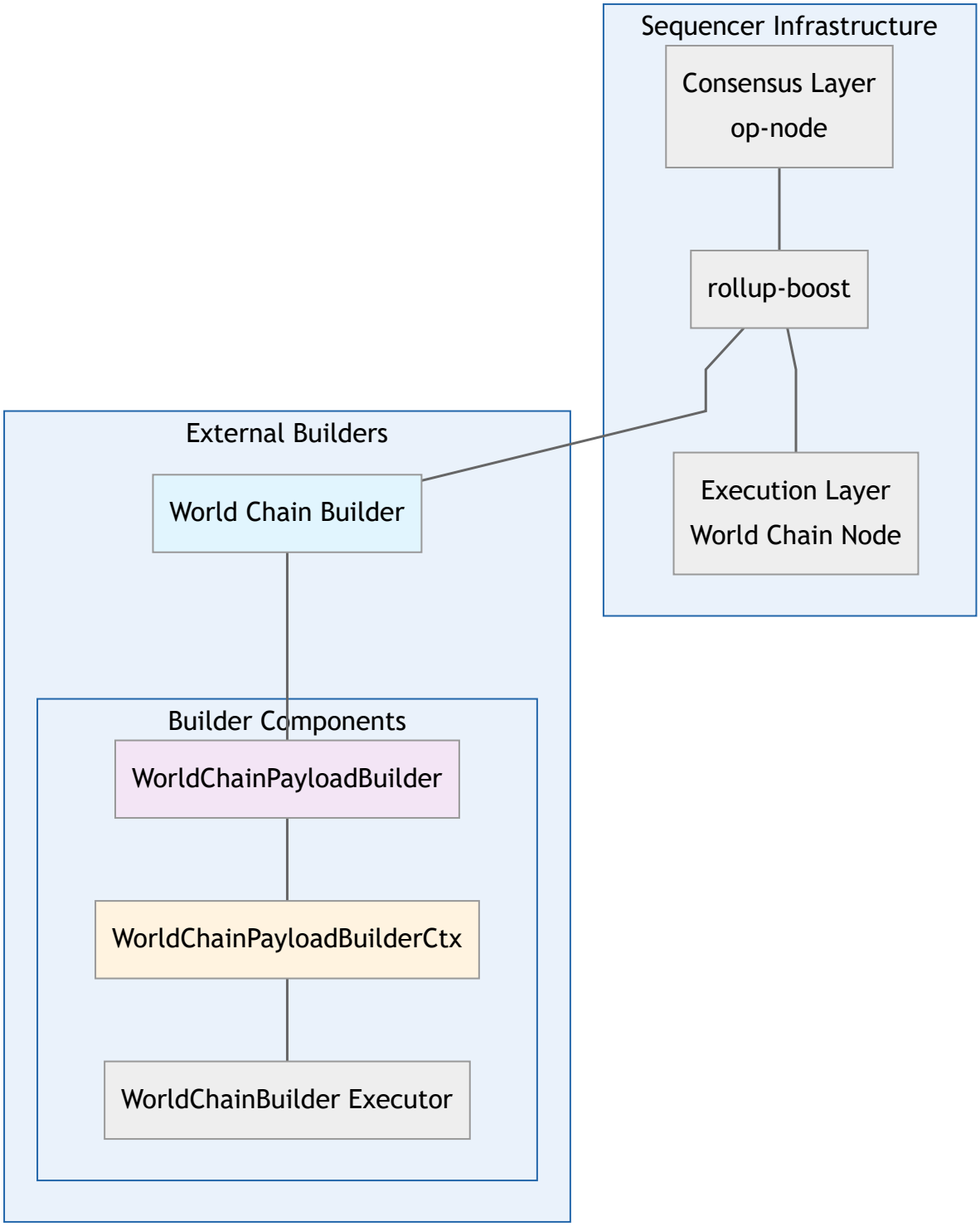
이러한 컨텍스트 시스템을 통해 World Chain은 다양한 운영 환경에서 최적화된 성능을 제공할 수 있습니다.

2.2 World Chain Builder 아키텍처 분석

World Chain Builder의 역할과 위치

World Chain Builder는 rollup-boost 생태계 내에서 외부 블록 빌더로 동작하며, PBH(Priority Blockspace for Humans) 메커니즘을 구현하는 핵심 구성 요소입니다.

시스템 내 위치



외부 블록 빌더로서의 동작 방식

World Chain Builder는 표준 Engine API를 통해 시퀀서와 통신하며, 다음과 같은 구조로 구성됩니다:

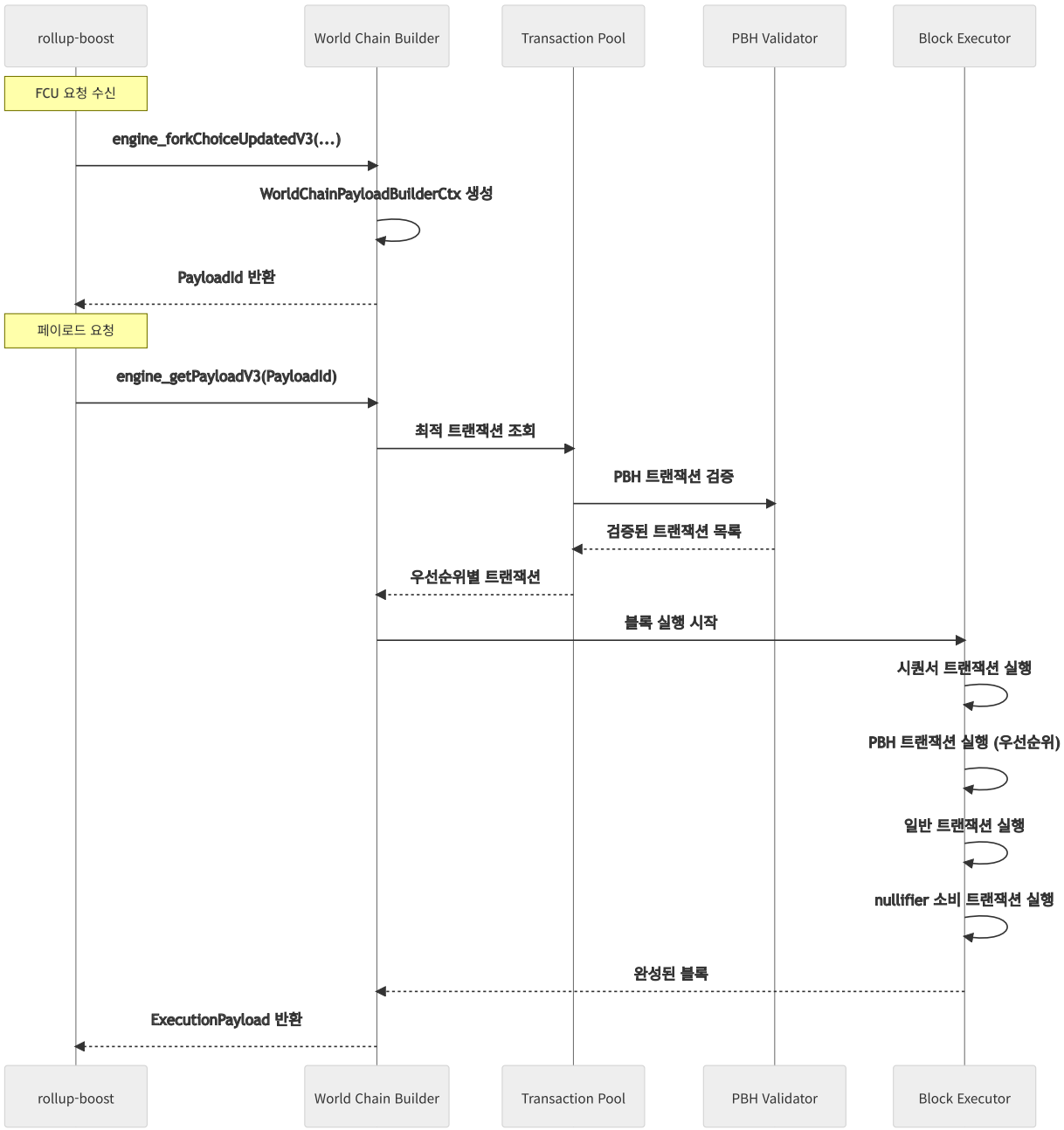
1. WorldChainPayloadBuilder 구조

```
pub struct WorldChainPayloadBuilder<Client, S, Txs = ()> {
```

```
// OP Stack 기본 페이로드 빌더
pub inner: OpPayloadBuilder<WorldChainTransactionPool<Client, S>, Client, OpEvm
// PBH 특화 설정들
pub verified_blockspace_capacity: u8, // PBH 블록스페이스 용량 (%)
pub pbh_entry_point: Address, // PBH EntryPoint 컨트랙트 주소
pub pbh_signature_aggregator: Address, // PBH 서명 집계자 주소
pub builder_private_key: PrivateKeySigner, // 빌더 개인키
}
```

2. 페이로드 빌드 프로세스

World Chain Builder의 블록 생성 과정은 다음과 같습니다:


rollup-boost
World Chain Builder
Transaction Pool
PBH Validator
Block Executor
FCU 요청 수신
engine_forkChoiceUpdatedV3(...)
WorldChainPayloadBuilderCtx 생성
PayloadId 반환
페이로드 요청
engine_getPayloadV3(PayloadId)
최적 트랜잭션 조회
PBH 트랜잭션 검증
검증된 트랜잭션 목록
우선순위별 트랜잭션
블록 실행 시작
시퀀서 트랜잭션 실행
PBH 트랜잭션 실행 (우선순위)
일반 트랜잭션 실행
nullifier 소비 트랜잭션 실행
완성된 블록
ExecutionPayload 반환
rollup-boost
World Chain Builder
Transaction Pool
PBH Validator
Block Executor

3. PBH 블록스페이스 관리

World Chain Builder는 다음과 같은 방식으로 PBH 블록스페이스를 관리합니다:

```
// PBH 전용 가스 한도 계산
let  verified_gas_limit = (self.verified_blockspace_capacity as u64 * gas_limit)


// PBH 트랜잭션 처리 로직
if  let Some(payloads) = pooled_tx.pbh_payload() {
// PBH 가스 한도 내에서만 실행
if  info.cumulative_gas_used + tx.gas_limit() > verified_gas_limit {
best_txs.mark_invalid(tx.signer(), tx.nonce());
continue;
}
// nullifier hash 중복 검사
if  payloads.iter().any(|payload| !spent_nullifier_hashes.insert(payload.nullifi
best_txs.mark_invalid(tx.signer(), tx.nonce());
continue;
}
}
```

시퀀서와의 상호작용 과정

1. Engine API 통신

World Chain Builder는 표준 Engine API를 구현하여 시퀀서와 통신합니다:

- **engine_forkChoiceUpdatedV3**: 새로운 블록 빌드 요청을 수신합니다
- **engine_getPayloadV3**: 완성된 페이로드를 반환합니다
- **engine_newPayloadV3**: rollup-boost를 통한 블록 검증에 사용됩니다

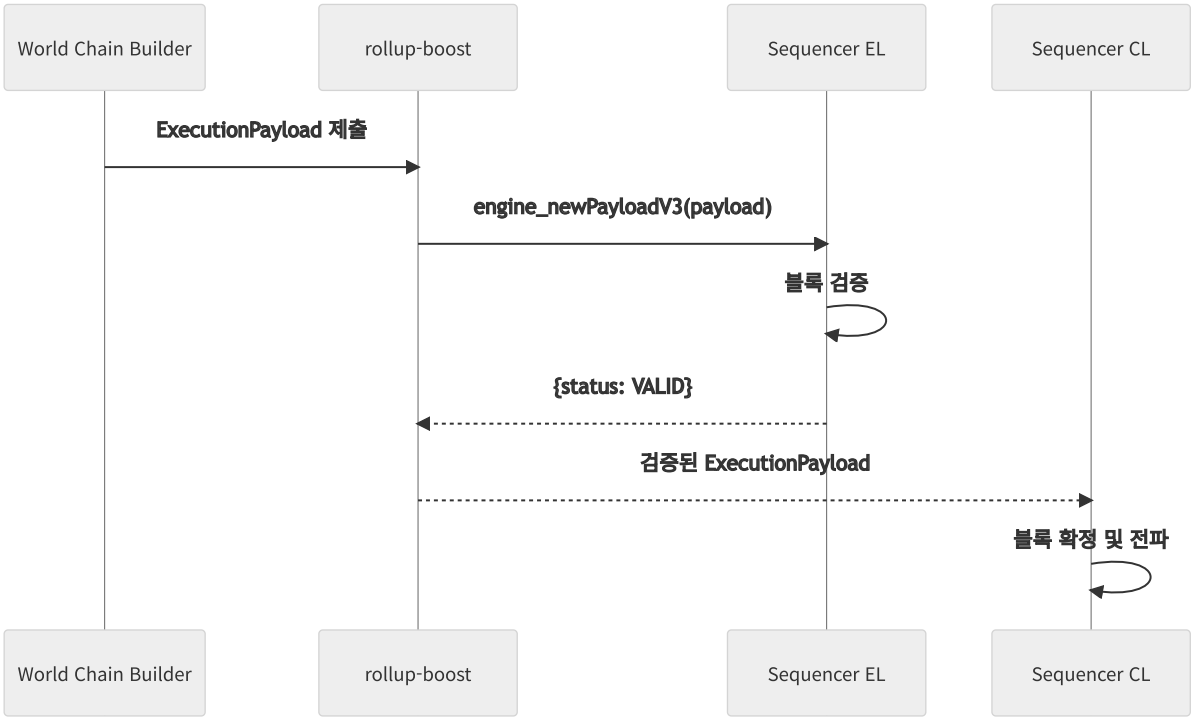
2. 트랜잭션 수신 및 처리

```
// 트랜잭션 풀에서 최적 트랜잭션 선택
let best_txs = best(ctx.best_transaction_attributes(builder.evm_mut().block()))

// PBH 우선순위에 따른 실행
ctx.execute_best_transactions(&mut info, &mut builder, best_txs, gas_limit)?
```

3. 블록 검증 및 제출

완성된 블록은 rollup-boost를 통해 시퀀서의 실행 클라이언트에서 검증됩니다:



4. 폴백 메커니즘

World Chain Builder가 응답하지 않거나 무효한 블록을 제출할 경우, rollup-boost는 자동으로 기본 실행 클라이언트의 블록으로 폴백합니다:

```
// 빌더 응답 시간 초과 또는 무효한 블록의 경우
if builder_response.is_timeout() || !builder_response.is_valid() {
// 기본 실행 클라이언트의 블록 사용
return default_execution_client_payload;
}
```

고성능 최적화 기능

1. Flashblocks 통합

World Chain Builder는 flashblocks 기술을 통해 고성능 블록 빌딩을 지원합니다:

```
pub struct FlashblocksPayloadBuilder<Pool, Client, CtxBuilder, Txs> {
pub evm_config: OpEvmConfig,
pub pool: Pool,
pub client: Client,
pub config: OpBuilderConfig,
pub best_transactions: Txs,
pub ctx_builder: CtxBuilder,
}
```

2. 상태 캐싱 및 재사용

- **캐시된 읽기**: 이전 블록 빌드의 상태 읽기를 재사용합니다
- **점진적 업데이트**: 새로운 트랜잭션만 추가하여 블록을 업데이트합니다
- **병렬 처리**: 여러 블록을 동시에 구성할 수 있습니다

3. 가스 최적화

```
// nullifier 소비를 위한 동적 가스 계산
pub const fn dyn_gas_limit(len: u64) -> u64 {
FIXED_GAS + len * COLD_SSTORE_GAS
}
```

```
// PBH 트랜잭션 실행 시 가스 조정

if  spent_nullifier_hashes.len() == payloads.len() {

gas_limit -= FIXED_GAS

}

gas_limit -= COLD_SSTORE_GAS * payloads.len() as u64;
```

결론

World Chain의 OP Stack 기반 아키텍처는 Optimism의 검증된 기술 스택을 활용하면서도 PBH라는 혁신적인 기능을 통해 인간 중심의 블록체인 경험을 제공합니다. World Chain Builder는 외부 블록 빌더로서 rollup-boost와 완벽하게 통합되어 있으며, 고성능 블록 빌딩과 PBH 우선순위 메커니즘을 통해 네트워크의 탈중앙화와 성능 향상에 기여합니다.